KV Forge: Mastering LLM Inference, from KV Cache to Production — Intermediate

Portfolio projects · lessons 10–25

Each takes 1–3 days and ends with something to show an employer.

Project 1: Brightside: size the KV budget and tune the engine to a TTFT target

Goal: Predict, then measure, how many tokens and concurrent requests one GPU serves for the Brightside support desk, and tune the engine until p95 time to first token meets a target you set, with each flag justified by a number. This is the running example's step 2 in app/.

Uses lessons: 03, 05, 06, 07, 08, 10, 14, 16, 18

1. Read the Llama 3.1 8B fields (32 layers, 8 KV heads, head_dim 128) from data/model_configs.json and compute bytes per token in FP16 as 2 x layers x KV heads x head_dim x 2 bytes; confirm 128 KiB per token against lesson 03 and write the formula in NOTES.md.

2. Predict the KV token budget on one NVIDIA L4 (24 GB): 0.92 x 24 GB, minus about 16 GB of FP16 weights, minus a 2 GB activation reserve you choose; divide by 128 KiB per token and write the predicted token count before you run anything.

3. Start the engine with the lesson 16 flags (max_model_len 4096, max_num_seqs 64, gpu_memory_utilization 0.92, prefix caching on) and copy the effective values from the startup log into config/engine.yaml; note any value that differs from your prediction and why.

4. Replay data/support_trace.jsonl at 1, 4 and 8 requests per second for 3 minutes each, and record TTFT p50 and p95, time per output token and goodput against a TTFT p95 target of 1,000 ms that you set in config/slo.yaml.

5. Run a 2 x 2 sweep of prefix caching (on, off) and chunked prefill (on, off) at 4 requests per second, using policy_prompt.txt (about 1,500 tokens) as the shared prefix; report the prefix hit rate from the metrics endpoint and a table of at least four rows.

6. Add step 2 of app/GUIDE.md: read the engine settings from config/engine.yaml, expose the latency histograms on /metrics, and add at least five pytest tests with FastAPI's TestClient, then run them with one command.

7. Write the README with the final flags, the measured p95 TTFT for each row of the sweep, the predicted versus logged KV token budget, and a rollback trigger (for example, p95 TTFT above 1,000 ms for 5 minutes).

Deliverable: A GitHub repo with NOTES.md, config/engine.yaml, config/slo.yaml, the replay script, the app step 2 code and tests, and a README whose tables give the predicted and measured KV budget, p50 and p95 TTFT per load level, and the 2 x 2 sweep results.

Stretch: Find the largest request rate at which p95 TTFT stays under 1,000 ms, by stepping from 1 to 8 requests per second in increments of 0.5, and report it with the sample size per step.

Project 2: Brightside at 70B: pick the hardware, TP degree and replicas for an SLO

Goal: Size a 70B model for the Brightside support desk on paper and against a measured number, choose the cheapest GPU layout that fits and meets the SLO, and write the decision in a form a platform review board would accept. This is the running example at its second scale step.

Uses lessons: 03, 04, 05, 19, 20, 22, 23, 24, 25

1. Read the Llama 3.1 70B fields (80 layers, 8 KV heads, head_dim 128) from data/model_configs.json and compute KV bytes per token in FP16; confirm 320 KiB per token against lesson 03.

2. Compute the weight memory of 70B in three formats: FP16 (2 bytes per parameter, about 140 GB), FP8 (1 byte) and 4-bit (0.5 byte); show that none fits one L4 (24 GB) and state the smallest TP degree that fits for each format.

3. Build a table with one row per option (L4 at TP 4 and 8, A100 80 GB at TP 2 and 4, H100 80 GB at TP 2), with columns for per-GPU weights, per-GPU KV tokens left after a 0.92 utilisation budget and a 2 GB activation reserve, and max concurrent requests at 1,800 tokens per request.

4. Estimate the all-reduce cost per decode step for your two best rows: two all-reduces per layer, each moving batch x 8,192 x 2 bytes, and compare the time with PCIe and NVLink bandwidth from lesson 22; say which options are plausible without NVLink.

5. Enter one GPU-hour price for each option from a pricing page you open that day, write the date and URL next to it, and compute cost per million output tokens at 20, 50 and 80 percent utilisation; if you have no GPU, use the recorded result in data/recorded/ and mark every measured number as recorded.

6. Choose one option and write a decision record of at most two pages: hardware, TP degree, replica count for 10 requests per second with 300 output tokens each, the memory and communication cost, cost per million tokens, and what would change the decision.

Deliverable: A repo with a calc.py that prints the tables from the arithmetic (so the numbers can be rerun), a price file with dates and sources, and a decision record whose tables give per-option weights, KV tokens, concurrency, all-reduce time and cost per million tokens.

Stretch: Add a mixed-precision option (FP8 weights with FP8 KV) and compute the concurrency gain it gives on your chosen GPU, checking the number against lesson 21's quality gate before you recommend it.

Project 3: Insurance claims summariser: a batch job that must finish overnight

Goal: Build an overnight job that turns 4,000 made-up insurance claim notes into structured JSON summaries in under two hours on one GPU, tuned for throughput rather than latency and gated on JSON validity. A different domain and a different SLO: the job cares about tokens per second and finishing time, not interactive TTFT.

Uses lessons: 04, 05, 10, 11, 12, 13, 20, 21

1. Write a seeded script that makes 200 made-up claim notes of 1,500 to 2,500 tokens each (labelled MADE UP in the file header), and a 50-note regression set with a reference JSON per note (claim_id, cause, amount_band, action).

2. Compute the batch budget: 200 requests of 2,200 tokens need 200 x 2,200 x 128 KiB of KV for an 8B model if all run at once; show that this does not fit an L4 and derive the max_num_seqs you can try (16, 32, 64) from the KV budget at 0.92 utilisation.

3. Run 200 notes at max_num_seqs 16, 32 and 64, three repeats each, with chunked prefill on and off, and record output tokens per second and median TTFT in a table of at least six rows.

4. Count preemption events in the 64-sequence runs from the server log (lesson 13) and drop any setting that preempts more than once per 100 requests, even if it is faster.

5. Quantize the weights to a 4-bit format with LLM Compressor or the recorded artefact (lesson 20), run the 50-note regression set on both variants, and report JSON validity, field exact match and tokens per second, with a 95 percent interval on the difference in JSON validity.

6. Write the batch runner: read the notes in batches equal to max_num_seqs, call the server's OpenAI-compatible endpoint, validate each reply against the JSON schema, retry a failed note once, and write results to a JSONL file; then time a full 4,000-note run (or extrapolate from 200 notes and say so) and check it finishes in under 2 hours.

Deliverable: A repo with the seeded note generator, the regression set, the batch runner, the sweep results and a README that gives the chosen config, tokens per second, the projected finish time for 4,000 notes, the JSON validity and field exact match for both weight variants with the interval, and a rollback rule (keep the BF16 weights if validity drops by more than 2 points).

Stretch: Add a second summary schema for a different claim type and show that the same engine config still meets the finish-time target, or say which setting you must change.

Project 4: Multi-tenant helpdesk SaaS: one base model, 40 adapters, shared prefixes

Goal: Plan the memory and cache layout for a SaaS helpdesk where 40 customers each have their own system prompt and LoRA adapter on one base model, and measure what sharing the base model and the prefix cache saves. The question a platform team is asked most: can we add tenants without adding GPUs?

Uses lessons: 05, 10, 11, 14, 15, 17

1. Write a seeded tenant mix for 40 made-up tenants: each has a system prompt of 1,200 to 1,800 tokens, and the top 5 tenants send 60 percent of 10,000 requests (labelled MADE UP in the file header).

2. Simulate the KV block allocator from lesson 11 with 16-token blocks for 200 concurrent requests on one L4 at 0.92 utilisation; report the blocks used, the free blocks and the internal fragmentation, and compare with reserving the maximum length per request (lesson 10).

3. Measure the prefix hit rate for 10,000 requests in two layouts: one shared global prompt, and one prompt per tenant; report hit rate per layout and per tenant group, using the metrics endpoint or your own counter.

4. Vary the unique question suffix from 50 to 800 tokens in steps of 150 and record the hit rate and TTFT for each, to find where caching stops helping.

5. Compare memory for 40 full 8B FP16 copies (about 16 GB each) against one base model plus 40 rank-16 LoRA adapters; compute the adapter size from the layer shapes in data/model_configs.json and show the total in GB for each layout.

6. Write a one-page decision: which tenants share a prefix group, how many tenants one L4 can host at the measured hit rate, and a note that sharing a cached prefix across tenants can leak timing, to be fixed in the security lesson later in the course.

Deliverable: A repo with the tenant generator, the allocator simulation, the prefix-cache runs, the adapter memory calculation and a README with the measured hit rates, the suffix-length curve, the memory table and the tenant-per-GPU answer with its evidence.

Stretch: Add a fairness rule: no tenant may take more than 30 percent of the KV blocks at any time, and show the p95 TTFT of the other 39 tenants before and after the rule.